VIEWFLIX - Database Management System Report

Movie & TV Streaming Platform - COMP2004 Term Project

Defne Kaya - Goktug Karaca | Instructor: Pinar Efe | June 2026

1. Introduction

VIEWFLIX is a relational database system designed for an online movie and TV streaming platform similar to Netflix. The system stores and manages user accounts, subscription plans, movies and series, genres, cast and crew, and all user interactions such as watch history, favorites and reviews. The whole project focuses on the database layer: it is built on MySQL and demonstrates a clean, fully normalized relational design rather than a front-end application. The database guarantees data integrity through primary and foreign keys and removes redundancy by following Third Normal Form (3NF).

2. Project Objective

The main objective of this project is to design and implement a database system that:

- Minimizes data redundancy through normalization (up to 3NF).
- Ensures data integrity using primary keys, foreign keys and integrity constraints.
- Efficiently handles many-to-many relationships using bridge (associative) tables.
- Supports scalable and consistent data management for a streaming platform.
- Provides meaningful DML operations and SQL queries that answer real business questions.

3. System Analysis and Requirements

After analysing how a streaming platform works, the VIEWFLIX database is required to support the following functional requirements:

- User registration and account management (active, suspended, cancelled).
- Subscription plan management (Basic, Standard, Premium) and tracking of active subscriptions and payment status.
- Storage of movies and TV series under a single, unified content model.
- Categorization of content using genres (a content item can have many genres).
- Representation of cast and crew information: actors, directors, writers and producers.
- Tracking of user watch history and viewing progress.
- Allowing users to mark content as favorites.
- Allowing users to submit ratings and written reviews for content.

4. Database Design Approach

A relational model is used in which each entity is represented as a table, and relationships between tables are implemented using foreign keys. Many-to-many relationships cannot be represented directly in a relational database, so they are resolved using bridge (associative) tables. In VIEWFLIX these are:

- Content_Genres - resolves the many-to-many relationship between Content and Genres.
- Content_Cast - resolves the many-to-many relationship between Content and People (with a role).
- Favorites - resolves the many-to-many relationship between Users and Content.

5. Normalization (3NF)

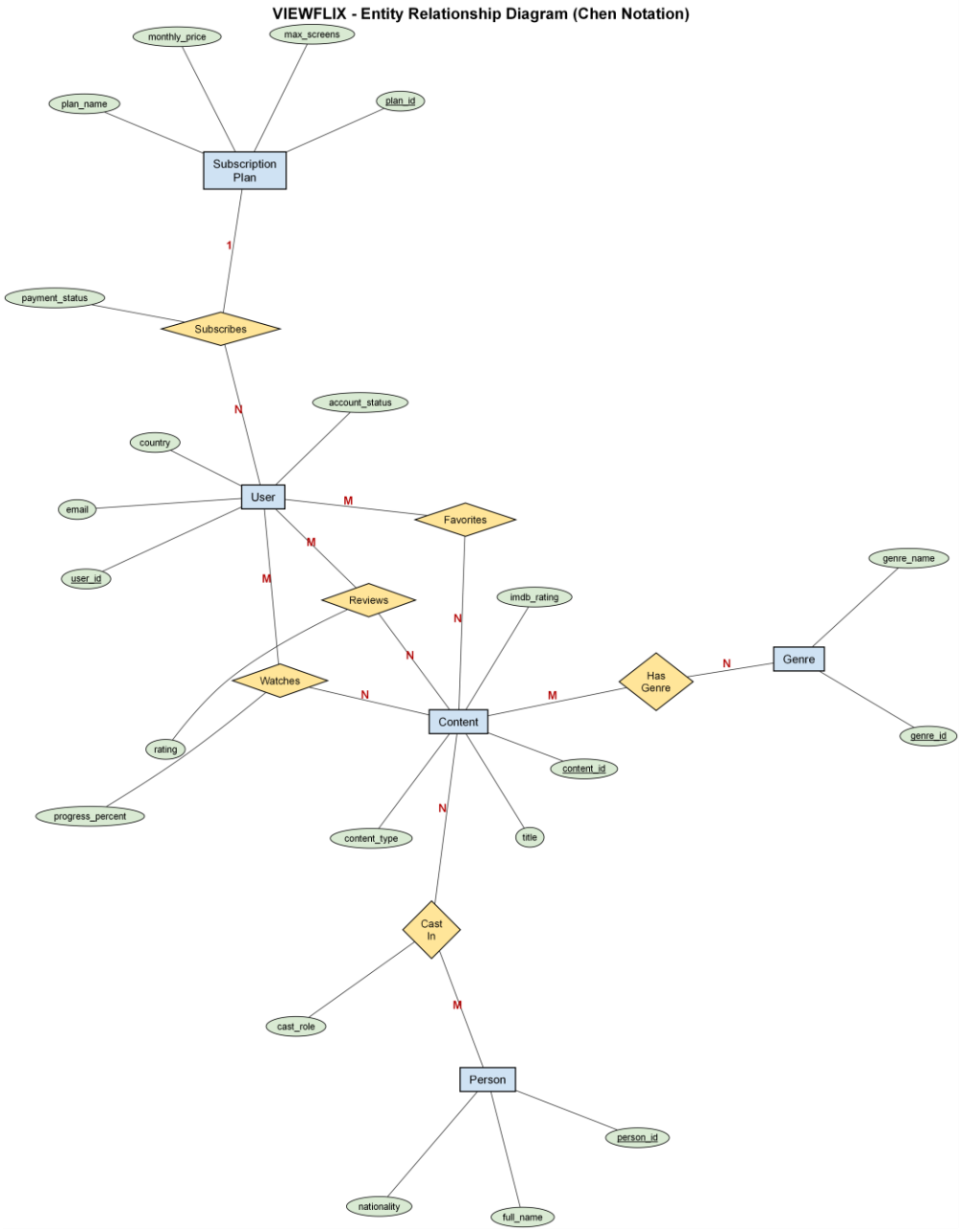
The database is designed in Third Normal Form to eliminate redundancy and update, insert and delete anomalies.

First Normal Form (1NF): All attributes are atomic and there are no repeating groups. Multi-valued data such as the genres of a title are moved to a separate table (Content_Genres) so that each field holds a single value.

Second Normal Form (2NF): The schema is in 1NF and every non-key attribute is fully functionally dependent on the whole primary key. Composite-key tables such as Content_Genres and Content_Cast contain no partial dependencies.

Third Normal Form (3NF): The schema is in 2NF and there are no transitive dependencies between non-key attributes; each attribute depends only on the primary key of its table. As a result the design removes redundancy and keeps the data consistent across all tables.

6. Entity Relationship Diagram



Chen notation: rectangles = entities, ellipses = attributes (primary keys underlined), diamonds = relationships, with cardinalities (1, N, M).

7. Entities, Keys and Constraints

Strong entities have their own single-column primary key. Weak / associative entities are identified by a composite primary key made of the foreign keys they connect.

Strong Entity	Primary Key

Users	user_id
Subscription_Plans	plan_id
Subscriptions	subscription_id
Content	content_id
Genres	genre_id
People	person_id
Watch_History	history_id
Reviews	review_id

Weak / Associative Entity	**Composite Primary Key**
Content_Genres	(content_id, genre_id)
Content_Cast	(content_id, person_id, cast_role)
Favorites	(user_id, content_id)

Primary keys uniquely identify each record, while foreign keys maintain relationships and enforce referential integrity. For example, each subscription references a valid user and a valid plan, and each review references a valid user and content. Additional integrity constraints are used: UNIQUE on users.email and reviews(user_id, content_id); NOT NULL on required columns; and CHECK constraints such as rating BETWEEN 1 AND 10, progress_percent BETWEEN 0 AND 100, and content_type IN ('Movie','Series'). ON DELETE CASCADE keeps the data consistent when a parent row is removed.

8. SQL Implementation - DML Operations

The project includes meaningful INSERT, UPDATE and DELETE operations. Examples:

```
UPDATE users SET account_status='Suspended' WHERE user_id=6;
UPDATE subscriptions SET status='Cancelled' WHERE subscription_id=6;
UPDATE reviews SET rating=9 WHERE review_id=10;
UPDATE watch_history SET progress_percent=100, is_completed=TRUE WHERE history_id=5;
DELETE FROM favorites WHERE user_id=1 AND content_id=2;
DELETE FROM reviews WHERE review_id=20;
```

9. SQL Implementation - Queries

Fifteen queries are provided in three groups: 5 simple (filtering/sorting), 5 complex (JOIN and subquery) and 5 aggregation (GROUP BY / HAVING). Selected examples, each answering a business question:

Q1 (simple) - List all movies ordered by IMDb rating:

```
SELECT title, release_year, imdb_rating FROM content
WHERE content_type='Movie' ORDER BY imdb_rating DESC;
```

Q2 (aggregation) - Number of titles and average IMDb rating per content type:

```
SELECT content_type, COUNT(*) AS title_count, ROUND(AVG(imdb_rating),2) AS avg_imdb
FROM content GROUP BY content_type;
```

Q3 (complex - subquery) - Content rated above the overall average:

```
SELECT title, content_type, imdb_rating FROM content
WHERE imdb_rating > (SELECT AVG(imdb_rating) FROM content) ORDER BY imdb_rating DESC;
```

Q4 (complex - join) - Average rating and number of reviews per title:

```
SELECT c.title, COUNT(r.review_id) AS review_count, ROUND(AVG(r.rating),2) AS avg_rating
FROM content c JOIN reviews r ON c.content_id=r.content_id
GROUP BY c.content_id, c.title ORDER BY avg_rating DESC;
```

Q5 (aggregation - HAVING) - Genres associated with more than one title:

```
SELECT g.genre_name, COUNT(cg.content_id) AS title_count
FROM genres g JOIN content_genres cg ON g.genre_id=cg.genre_id
GROUP BY g.genre_id, g.genre_name HAVING COUNT(cg.content_id) > 1 ORDER BY title_count DESC;
```

10. Conclusion

VIEWFLIX successfully models a streaming platform using a fully normalized relational structure. By applying 3NF and proper relational design with primary keys, foreign keys and integrity constraints, the system ensures data integrity, reduces redundancy and supports scalable, consistent data management. The database has been implemented and tested on MySQL, where all tables, sample data, DML operations and queries run successfully, and is ready for a live demonstration.